

Exact Circular Pattern Matching Using the BNDM Algorithm

K. H. Chen, G. S. Huang* and R. C. T. Lee

Department of Computer Science and Information Engineering,
National Chi Nan University, Puli, Nantou, Taiwan.
{s95321902, shieng, rctlee}@ncnu.edu.tw

ABSTRACT

In this paper, we discuss a problem that performs searching a circular string on text from bit-parallelism. Given a string $X = x_1x_2 \cdots x_m$, a string $X^i = x_ix_{i+1} \cdots x_mx_1 \cdots x_{i-1}$, for $1 \leq i \leq m$, is a circular string of X . Given a text string $T = t_1t_2 \cdots t_n$ and a pattern P , the circular pattern matching problem is to find all occurrences of P^i in the text T . Our algorithm only uses the composition of bitwise logical operations and basic arithmetic operations, and applies this technique to solve the problem. The algorithm is given name CBNDM. Our idea is based on the BNDM algorithm. We give some experiments to verify that it has good behavior for random strings.

1 Introduction

In this paper, we are interested in the following problem. Given a string $X = x_1x_2 \cdots x_m$, a string $X^i = x_ix_{i+1} \cdots x_mx_1 \cdots x_{i-1}$, for $1 \leq i \leq m$, is a circular string of X [16]. Given a text string $T = t_1t_2 \cdots t_n$ and a pattern P , the circular pattern matching problem is to find all occurrences of P^i in the text T . Throughout the paper, given $A = a_1a_2 \cdots a_n$, we use $A(i, j)$ to denote $a_ia_{i+1} \cdots a_j$ (notice that if $j < i$, it does mean $A(i, n)A(1, j)$).

There exists a linear time algorithm to determine whether T is a circular rotation of P if T and P have the same length and T consists of a suffix of P followed by a prefix of P (see [8] for details). In our case, we further consider the length n of T and the length m of P are different such that P is a circular pattern and $m < n$. It implies the algorithm must have *shift strategy* in each attempt.

For example, assume that $T = \text{acgttaccat}$ and $P = \text{cctta}$. We then find that a circular pattern $P^3 = \text{ttacc}$ occurs in $T(4, 8)$. Some special case applications of this problem, for example, in image processing analysis, or in bacterial and mitochondrial DNA analysis can be found in [8].

To solve this problem, a simple method is to transform the problem to the multiple pattern matching problem. For a pattern P of length m , we produce m patterns $P^1, P^2, \dots, P^m$ from the original pattern P . For example, in the above example, $P = \text{cctta}$. We have $P^1 = \text{cctta}$, $P^2 = \text{cttac}$, $P^3 = \text{ttacc}$, $P^4 = \text{tacct}$, $P^5 = \text{acctt}$.

We then can solve it by some proposed multiple-pattern matching algorithms [1, 2, 3, 4, 5, 7, 9, 10, 12, 13, 15, 17]. However, we like to point out that our problem is not a general case of the multiple pattern matching problem since in a general case multiple pattern matching problem, the patterns are not inter-related while in our case, the patterns are inter-related. Therefore, we shall point out that we can use this special property to design an efficient algorithm for our purpose.

Our algorithm is based upon a bit-parallel technique, called BNDM (Backward Nondeterministic Dawg Matching) algorithm [13]. It belongs to factor-based algorithm [14]. Therefore, we use the reverse factor algorithm [6] to introduce the basic idea of the factor-based approach. We shall show that the BNDM algorithm can be modified to suite our problem. We call our algorithm the CBNDM (Circular BNDM) algorithm.

The organization of this paper is as follows. We first introduce the basic idea of the reverse factor algorithm and using bit-parallel technique which is called the BNDM algorithm for single pattern case in Sect. 2 and Sect. 3 respectively. Section 4 introduces how to use the multiple version of the BNDM algorithm which is called the multiple BNDM algorithm to solve the circular pattern matching problem. In Sect. 5, we propose

*To whom correspondence should be addressed.
Email:shieng@ncnu.edu.tw

our algorithm for circular pattern case. The time complexity is analyzed in Sect. 6. We also provide several useful experimental results in Sect. 7. Finally, we conclude this work in Sect. 8.

2 The Single Pattern Matching Solved by the Reverse Factor Algorithm

The reverse factor algorithm was proposed in [6]. A comprehensive discussion of the algorithm can be found in [11]. Before introducing the algorithm, we need to define a term, called proper suffix. A proper suffix of a string A is a suffix of A which is not exactly A . The basic principles of this algorithm can be illustrated as follows:

1. Given a text string T and a pattern string P where $|P| = m$, we open a window W with size m .
2. Let $LSP(W, P)$ ($LPSP(W, P)$) denote the longest (proper longest) suffix of W which is equal to a prefix of P . If $|LSP(W, P)| < m$, we shift $m - |LSP(W, P)|$ steps. Otherwise, $|LSP(W, P)| = m$ and an exact match is found. We report this and shift $m - |LPSP(W, P)|$ steps.

Consider the case where $T = \text{acgtttga}$ and $P = \text{tttga}$. The first window of $T = \text{acgtttga}$ will be $W = \text{acgtt}$ as shown below:

$$\begin{aligned} T &= \text{acgtttga} \\ P &= \text{tttga} \end{aligned}$$

We can see that $LSP(W, P) = \text{tt}$. We shift the pattern so that the prefix tt of $P = \text{tttga}$ is aligned with the suffix tt of $W = \text{acgtt}$ as shown below:

$$\begin{aligned} T &= \text{acgtttga} \\ P &= \quad \text{tttga} \end{aligned}$$

Now, an exact match is found.

To find $LSP(W, P)$, we may build the suffix tree of the reverse of P in the preprocessing phase and scan the window from right to left in searching phase. However, the suffix tree approach to solve the reverse factor algorithm has a problem that it is not easy to implement. Therefore, Navarro and Raffinot proposed a bit-parallelism approach to simplify the reverse factor algorithm which is called the BNDM algorithm [13].

3 The Single Pattern Matching Solved by the BNDM Algorithm

For succinctness, we use the following notations in the bit-parallel technique. Exponentiation stands for bit repetition (e.g. $0^31=0001$). The window which we open is denoted as $W = w_1w_2 \cdots w_m$. In the bit operators, “&” represents bitwise-AND operation. The shift left operation, “ $\ll$ ” moves the bits to the left and enters zeros from the right.

To use the bit-parallel approach, we first define an incidence matrix B of size r by m where r is the alphabet size. $B[a]$ is a row of B that corresponds to the letter a which is set to be 1 at i -th position if and only if $p_i = a$. We call $B[a]$ a *mask*. For example, assume that $P = \text{acggt}$. We then have the masks $B[a] = (10000)$, $B[c] = (01000)$, $B[g] = (00110)$ and $B[t] = (00001)$. For the j -th iteration, we read w_{m-j+1} for $1 \leq j \leq m$. Let $D_j = d_{1,j}d_{2,j} \cdots d_{m,j}$ be a bit vector. The bit $d_{i,j}$ is set to 1 if and only if $p_i \cdots p_{i+j-1} = w_{m-j+1} \cdots w_m$ for some j . Let $W = \text{acgg}$ and $P = \text{ggca}$. Then $D_2 = (1000)$ because $p_1p_2 = \text{gg} = w_3w_4$. Therefore, if $d_{1,2} = 1$, it means that $p_1 \cdots p_j = w_{m-j+1} \cdots w_m$ and we have found a prefix of P which is equal to a suffix of W . This particular information is very important. We shall use a parameter *last* to record this information. That is, if $d_{1,j} = 1$ after the j -th iteration, $\text{last} = j$.

Initially, we set $D_0 = 1^m$ and $\text{last} = 0$. We scan the window W from right to left. When a new character in W was scanned at the j -th iteration, we set D'_j based on the following rule:

$$D'_j = (D_{j-1} \& B[w_{m-j+1}]) \quad (1)$$

If $D'_j = 0^m$, for some j , we stop this attempt. Then $|LSP(W, P)| = \text{last}$ and we shift $S = m - \text{last}$ steps.

If $D'_j \neq 0^m$ and $j < m$, we check whether $d_{1,j} = 1$. We set last to be j if $d_{1,j} = 1$ and take no action if otherwise. After checking whether $d_{1,j} = 1$, we shift vector D'_j one bit to the left, fill the rightmost bit to be 0 and store this value to D_j . For example, assume that $D'_j = (0101)$, after shifting, we have $D_j = (1010)$. This is denoted as

$$D_j = D'_j \ll 1 \quad (2)$$

and continue the next iteration.

If $D'_j \neq 0^m$ and $j = m$, it means that we have found an exact match of P in T and $|LSP(W, P)| = m$. We then stop the attempt,

report a match and shift $S = m - \text{last}$ steps. Algorithm 1 shows a complete description of the single pattern matching using the BNBM algorithm. Notice that in Algorithm 1, we omit the subscript j for all variables such as D_j and $d_{i,j}$ for the purpose to re-use memory.

Algorithm 1: The single pattern matching using the BNBM algorithm

Input: Two strings $T = t_1 \dots t_n$ and $P = p_1 \dots p_m$
Output: The locations of all occurrences of P in T

```

1 Begin
2   Set  $i = m$ .
3   Compute the incidence matrix  $B$  of  $P$ .
4   While  $i \leq n$  do
5      $j = 0$  and  $\text{last} = 0$ .
6      $D = 1^m$ .
7      $W = T(i - m + 1, i)$ .
8     While  $D \neq 0^m$  do
9        $j = j + 1$ .
10       $D = D \& B[w_{m-j+1}]$ .
11      If  $D = 0^m$  then exit.
12      If  $j = m$  and  $d_1 = 1$  then report an
        occurrence of  $P$  at position  $i - m + 1$  of
         $T$  and exit.
13      If  $j < m$  and  $d_1 = 1$  then  $\text{last} = j$ .
14       $D = D \ll 1$ .
15    End of while
16     $i = i + (m - \text{last})$ .
17  End of while
18 End
```

For example, we assume that $T = \text{acggtgac}$, $P = \text{tgac}$. We begin with $T = \text{acggtacc}$, $D_0 = (1111)$, $B[a] = (0010)$, $B[c] = (0001)$, $B[t] = (1000)$, $B[g] = (0100)$, $m = 4$, $n = 8$, $\text{last} = 0$.

In the first attempt, the first window is $W = \text{acgg}$. In the first iteration, we have $D'_1 = (1111) \& B[g] = (1111) \& (0100) = (0100)$. After left-shifting of D'_1 once, we have $D_1 = (1000)$ and $\text{last} = 0$.

In the second iteration, we found that $D'_2 = (1000) \& (0100) = (0000)$ and $\text{last} = 0$. Therefore, $|LSP(W, P)| = 0$. We then shift the window $S = 4 - 0 = 4$ steps right. The processing of the first attempt is shown in Figure 1.

In the second attempt, we have $W = \text{tgac}$. We found that $j = 4$ and $d_1 = 1$ in the final iteration of this attempt. It means that we have found an occurrence of P in $T(5, 8)$. We report a match and get $|LSP(W, P)| = 0$ because $\text{last} = 0$. The shift $S = 4 - 0 = 4$. Because the end position of the window has exceeded the end of the text, we stop searching. The processing of the second attempt is shown in Figure 2.

4 The Circular Pattern Matching Problem Solved by the MultiBNBM Algorithm

We can reduce the circular pattern matching problem to the multiple matching problem, by expanding the circular pattern P into ordinary patterns $P^1, \dots, P^m$ and then checking their occurrences in the text. In the following, we shall introduce MultiBNBM algorithm [13, 14], a multiple pattern matching algorithm which is based upon the reverse factor algorithm and uses the bit-parallel technique. It concatenates m patterns into a string $X = P^1 P^2 \dots P^m$ with length M where $M = m^2$. For example, let $P = \{\text{cct}, \text{aca}, \text{gtc}\}$. Then the concatenation of $X = \text{cctacagtc}$.

The preprocessing phase of the MultiBNBM algorithm is similar to the BNBM algorithm. The incidence matrix B with size r by M , the bit vector $D = d_1 d_2 \dots d_M$ and sentinel last are still used. In each attempt, it finds $\text{last} = \max(LSP(W, P^i))$, $1 \leq i \leq m$. Then shift $S = m - \text{last}$. Let us assume that $T = \text{acct}$. For the set of patterns, the length of each pattern is 3. We therefore set the window to be $W = T(1, 3) = \text{acc}$ with length 3. For each pattern P^i in $P = \{\text{cct}, \text{aca}, \text{gtc}\}$, it can be seen that $|LSP(W, P^1)| = 2$, $|LSP(W, P^2)| = 0$ and $|LSP(W, P^3)| = 0$. Shift $S = 3 - 2 = 1$.

As before, we will use a vector D_j . For the j -th iteration, we read w_{m-j+1} for $1 \leq j \leq m$. Let $D_j = d_{1,j} d_{2,j} \dots d_{m^2,j}$ be a bit vector with length m^2 bits. Consider the bit $d_{a,j}$ in D_j . Note that $a = m(k-1) + i$. Thus the bit $d_{a,j}$ in D_j is set to be 1 if and only if $p_i^k \dots p_{i+j-1}^k = w_{m-j+1} \dots w_m$ where $k = \lceil a/m \rceil$ and $i = a - m(k-1)$. Therefore, if $i = 1$ and $d_{m(k-1)+i,j} = d_{m(k-1)+1,j} = 1$ in D_j , this means that we have found a prefix $p_1^k \dots p_j^k$ of P^k which is equal to the suffix $w_{m-j+1} \dots w_m$ of W .

Let $W = \text{tcc}$ and $P = \{\text{cct}, \text{aca}, \text{gtc}\}$. For $j = 2$, we will find that $p_1^1 p_2^1 = \text{cc} = w_3 w_4$. In such a situation, we have $j = 2$, $i = 1$ and $k = 1$. Thus $a = m(k-1) + i = 3(1-1) + 1 = 1$ and $D_2 = (100000000)$.

Initially, we set $D_0 = 1^{m^2}$ and $\text{last} = 0$. We scan the window from right to left. When a new character in the window was scanned at the j -th iteration, we set D'_j based on Equation (1) in Sect. 3.

$$D'_j = (D_{j-1} \& B[w_{m-k+1}])$$

If $D'_j = 0^{m^2}$, for some j , we stop the scanning.

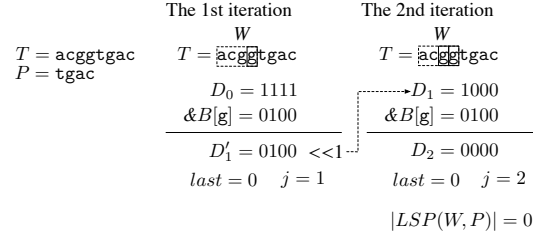


Figure 1: Illustration of the first attempt.

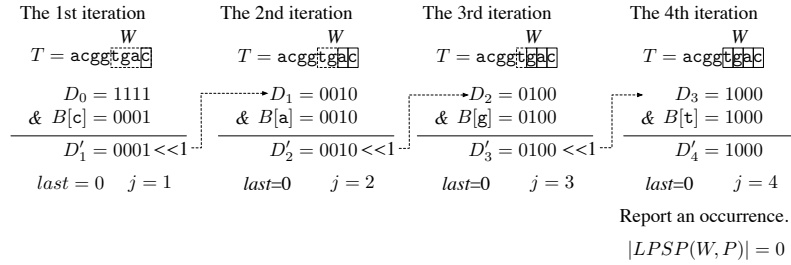


Figure 2: Illustration of the second attempt.

Then we shift $S = m - last$ steps.

Let $DF = (10^{m-1})^m$. If $D'_j \neq 0^{m^2}$ and $j < m$, we check whether $(D'_j \& DF) = 0^{m^2}$. Recall that DF has bit one in the highest bit position of each m -bit field, and zeros elsewhere. The operation $D'_j \& DF$ is to find out whether there is a prefix of P^k which is equal to a suffix of W , for $1 \leq k \leq m$. If $(D'_j \& DF) \neq 0^{m^2}$ and $d'_{m(k-1)+1,j} = 1$ for some k , it means that we have found a prefix of P^k with length j which is equal to a suffix of W . We set $last$ to be j if $(D'_j \& DF) \neq 0^{m^2}$ and take no action if otherwise. After checking whether $(D'_j \& DF) = 0^{m^2}$, we shift vector D'_j one bit to the left, fill the rightmost bit to be 0 for each m -bit field, and store the result to D_j . For example, assume that $D'_j = (001011001)$ and $m = 4$, after shifting, we have $D_j = (010100010)$.

There is one problem here. After left shifting, the highest bit corresponding to P^k will become the lowest bit of P^{k-1} . To solve this problem, we define a bit mask $CL = (1^{m-1}0)^m$ and perform the following operation:

$$D_j = (D'_j \ll 1) \& CL \quad (3)$$

If $D'_j \neq 0^{m^2}$ and $j = m$, it means that we have found an exact match of P^k for some k in T . We then report a match and shift $S = m - last$ steps.

Algorithm 2 shows a complete description for the circular pattern matching using the MultiBNDM algorithm.

If we use the MultiBNDM algorithm directly to solve the circular pattern matching problem, the time complexity is $O(nm \lceil m^2/\kappa \rceil)$ time and space-complexity is $O(m^2)$ where κ is the machine-word size [13, 14]. Therefore, we propose a new algorithm whose time complexity is $O(nm \lceil m/\kappa \rceil)$ time and space-complexity is $O(m)$ (see Sect. 6 for details). We call our algorithm the CBNDM (Circular Backward Nondeterministic Dawg Matching) algorithm which will be introduced in the next section.

5 The Circular Pattern Matching Problem Solved by the CBNDM Algorithm

In this section, we present our algorithm for solving the circular pattern matching problem. The algorithm is called CBNDM. The preprocessing phase of the CBNDM algorithm is the same as the BNDM algorithm for the single pattern matching.

To find this in a bit-parallel manner, we define an incidence matrix B_c with size r by m . $B_c[a]$ is set to be 1 at the i -th position if and only if $p_i = a$ for letter a in the alphabet. From our example where $P = \text{ggtca}$ and $W = \text{accag}$, $B_c[a] = (00001)$, $B_c[c] = (00010)$, $B_c[g] = (11000)$, $B_c[t] = (00100)$.

For the j -th iteration of a window, let $C_j =$

Algorithm 2: The circular pattern matching using the MultiBNDM algorithm

Input: A string T of length n and m patterns, $P^1, P^2, \dots, P^m$ rotated from the original pattern P respectively

Output: The locations of all occurrences of P^k for some k in T

```

1 Begin
2   Set  $i = m$ ,  $pos = m$  and  $M = m^2$ .
3   Set  $X = P^1 P^2 \dots P^m$ .
4   Compute the incidence matrix  $B$  of  $X$ .
5   Set  $CL = (1^{m-1}0)^m$ .
6   Set  $DF = (10^{m-1})^m$ .
7   While  $pos \leq n$  do
8      $j = 0$  and  $last = 0$ .
9      $D = 1^m$ .
10     $W = T(pos - m + 1, pos)$ .
11    While  $D \neq 0^m$  do
12       $j = j + 1$ .
13       $D = D \& B[w_{m-j+1}]$ .
14      If  $D = 0^m$  then exit.
15      If  $j = m$  and  $(D \& DF) \neq 0^m$  then
        report an exact match at  $t_{pos-m+1}$  and
        exit.
16      If  $j < m$  and  $(D \& DF) \neq 0^m$  then
         $last = j$ .
17       $D = (D \ll 1) \& CL$ .
18    End of while
19     $pos = pos + (m - last + 1)$ .
20  End of while
21 End

```

$c_1 c_2 \dots c_m$ be a bit vector. The bit c_i is set to be 1 if and only if there exists a prefix of circular string P^i which is of length j and equal to $w_{m-j+1} \dots w_m$, for some i and j . Let $W = \text{accag}$ and $P = \text{ggtca}$. Note that $P^4 = p_4 p_5 p_1 p_2 p_3 = \text{caggt}$. Then $C_3 = (00010)$ because $p_4 p_5 p_1 = \text{cag} = w_3 w_4 w_5$. Thus, we have found a prefix of P^4 which is equal to a suffix of W with length 3. To achieve this goal, we define the *left-rotation operation*, " $\llcorner$ " which rotates vector one bit left and fill its rightmost bit to be the first bit. For instance, if $C_j = (10010)$, $C_j \llcorner 1 = (000101)$. Recall that $C_j = c_{1,j} c_{2,j} \dots c_{m,j}$. After $C'_j = C_j \llcorner 1$ operation, we have a new bit vector $C'_j = c'_{1,j} c'_{2,j} \dots c'_{m,j} = c_2 c_3 \dots c_m c_1$.

Initially, we set $C_0 = 1^m$. We scan the window W from right to left. For the j -th iteration, we read w_{m-j+1} for $1 \leq j \leq m$. Let $LSP_c(W, P)$ be the longest suffix of the window which is equal to a circular factor of the circular pattern. When a new character in the window was scanned at the j -th iteration, we perform an AND operations of B_c and C_{j-1} based on Equation (4).

$$C_j = (C_{j-1} \llcorner 1) \& B_c[w_{m-j+1}] \quad (4)$$

If $C_j = 0^m$ for some j , we stop the scanning. Then $|LSP_c(W, P)| = j - 1$ and we shift the window $S = m - (j - 1)$ steps for the next attempt.

If $C_j = c_1 c_2 \dots c_m \neq 0^m$, $j < m$ and $c_i = 1$, it means that the suffix $w_{m-j+1} \dots w_m$ of W is equal to the prefix of P^i with length j and we continue the next iteration.

If $C_j \neq 0^m$ and $j = m$, it means that we have found an exact match of P^i in T and $|LSP_c(W, P)| = m$. We then stop the iteration, report a match and shift the window $S = 1$ step.

Recall that the example $P = \text{ggtca}$ and $W = \text{accag}$. In the first iteration, since the rightmost character $w_5 = \text{g}$, based on Equation (4), we perform a left rotation of $C_0 = (11111)$. We then obtain $C'_0 = (11111)$. After performing an AND operations of $B_c[\text{g}] = (11000)$ and $C'_0 = (11111)$, we have $C_1 = (11000)$.

Algorithm 3 shows a complete description of the CBNDM algorithm.

Algorithm 3: The circular pattern matching using the CBNDM algorithm

Input: Two strings T and P with length n and m respectively.

Output: The locations of all occurrences of $P^i = p_i p_{i+1} \dots p_m p_1 \dots p_{i-1}$ where $1 \leq i \leq m$ in T

```

1 Begin
2   Set  $pos = m$ .
3   Compute the incidence matrix  $B_c$  of  $P$ .
4   While  $pos \leq n$  do
5      $j = 0$ .
6      $C_0 = 1^m$ .
7      $W = T(pos - m + 1, pos)$ .
8     While  $C_j \neq 0^m$  do
9        $j = j + 1$ .
10       $C_j = (C_{j-1} \llcorner 1) \& B_c[w_{m-j+1}]$ .
11      If  $C_j = 0^m$  then exit.
12      If  $C_j \neq 0^m$ ,  $j = m$  and  $c_i = 1$  then
        report an exact match at position
         $pos - m + 1$  of  $T$  and exit.
13    End of while
14     $pos = pos + (m - j + 1)$ .
15  End of while
16 End

```

We give a complete example that $T = \text{atggtcac}$ and $P = \text{tcag}$. In the preprocessing phase, the incidence matrix B_c of P is shown as follows.

$$\begin{aligned}
B_c[\text{a}] &= (0010), \\
B_c[\text{c}] &= (0100), \\
B_c[\text{g}] &= (0001), \\
B_c[\text{t}] &= (1000).
\end{aligned}$$

Initially, $C_0 = 1^m$. In the first attempt, $j = 1$

$$\begin{array}{l}
P = \text{tcag} \quad j = 1 \quad W = \text{atgg} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array}
\quad
\begin{array}{l}
T = \text{atgg} \text{tcac} \\
\begin{array}{r}
C_0 = 1111 \xrightarrow{\ll} 1 \\
C'_0 = 1111 \\
\hline
\& B_c[g] = 0001 \\
\hline
C_1 = 0001
\end{array}
\end{array}$$

Figure 3: The first iteration in the first attempt.

$$\begin{array}{l}
P = \text{tcag} \quad j = 2 \quad W = \text{atgg} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array}
\quad
\begin{array}{l}
T = \text{atgg} \text{tcac} \\
\begin{array}{r}
C_1 = 0001 \xrightarrow{\ll} 1 \\
C'_1 = 0010 \\
\hline
\& B_c[g] = 0001 \\
\hline
C_2 = 0000
\end{array}
\end{array}$$

Figure 4: The second iteration in the first attempt.

and we scan the characters of window $W = \text{atgg}$ from right to left. In the first iteration, the rightmost character $w_{m-j+1} = w_{4-1+1} = 4$ is g and it is shown in Figure 3.

In Figure 3, after a bit-rotation operation and an AND operation, we found $C_1 = ((1111) \xrightarrow{\ll} 1) \& (0001) = (0001)$. In C_1 , $c_4 = 1$. Thus we have $j = 1$ and $i = 4$. This means that the suffix $w_{m-j+1} = w_4$ is equal to the prefix of $P^4 = \text{gtca}$ with length $j = 1$. We continue the next iteration to read the next character of W .

In the second iteration, $j = 2$, we have $C_1 = (0001)$ and $w_3 = g$. After calculating, we found that $C_2 = ((0001) \xrightarrow{\ll} 1) \& (0001) = (0000)$ containing all 0's. We stop the attempt. Note that $|LSP_c(W, P)| = j - 1 = 1$. Then the shift $S = 4 - 1 = 3$ steps, which is shown in Figure 4.

In the second attempt, initially $j = 1$ and we scan the characters of window $W = \text{gtca}$ from right to left. We start with the rightmost character $w_4 = a$. The processing for the first iteration of the second attempt is shown in Figure 5.

In Figure 5, we found that $C_1 = ((1111) \xrightarrow{\ll} 1) \& (0010) = (0010)$ based on

$$\begin{array}{l}
P = \text{tcag} \quad j = 1 \quad W = \text{gtca} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array}
\quad
\begin{array}{l}
T = \text{atgg} \text{tcac} \\
\begin{array}{r}
C_0 = 1111 \xrightarrow{\ll} 1 \\
C'_0 = 1111 \\
\hline
\& B_c[a] = 0010 \\
\hline
C_1 = 0010
\end{array}
\end{array}$$

Figure 5: The first iteration in the second attempt.

$$\begin{array}{l}
P = \text{tcag} \quad j = 2 \quad W = \text{gtca} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array}
\quad
\begin{array}{l}
T = \text{atgg} \text{tcac} \\
\begin{array}{r}
C_1 = 0010 \xrightarrow{\ll} 1 \\
C'_1 = 0100 \\
\hline
\& B_c[c] = 0100 \\
\hline
C_2 = 0100
\end{array}
\end{array}$$

Figure 6: The second iteration in the second attempt.

$$\begin{array}{l}
P = \text{tcag} \quad j = 3 \quad W = \text{gtca} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array}
\quad
\begin{array}{l}
T = \text{atgg} \text{tcac} \\
\begin{array}{r}
C_2 = 0100 \xrightarrow{\ll} 1 \\
C'_2 = 1000 \\
\hline
\& B_c[t] = 1000 \\
\hline
C_3 = 1000
\end{array}
\end{array}$$

Figure 7: The third iteration in the second attempt.

Equation (4). In C_1 , $c_3 = 1$. We have $j = 1$ and $i = 3$. This means that the suffix $w_{m-j+1} = w_4$ is equal to the prefix of $P^3 = \text{agtc}$ with length $j = 1$. We continue the next iteration.

In the second iteration, $j = 2$, we have $C_1 = (0010)$ and the character of w_3 is c . After computing, we found that the bits in $C_2 = ((0010) \xrightarrow{\ll} 1) \& (0100) = (0100)$ are not all 0's. In C_2 , $c_2 = 1$. Thus we have $j = 2$ and $i = 2$. This means that the suffix w_3w_4 is equal to the prefix of $P^2 = \text{cagt}$ with length $j = 2$. We continue the next iteration. The result is shown in Figure 6.

In the third iteration, $j = 3$, we have $C_2 = (0100)$ and the character $w_2 = t$. After computing, we found that the bits in $C_3 = ((0100) \xrightarrow{\ll} 1) \& (1000) = (1000)$ are not all 0's. In C_3 , $c_1 = 1$. Thus we have $j = 3$ and $i = 1$. This means that the suffix $w_2w_3w_4$ is equal to the prefix of $P^1 = \text{tcag}$ with length $j = 3$. The process for the third iteration in the second attempt is shown in Figure 7.

In the fourth iteration, $j = 4$, we have $C_3 = (1000)$ and the character $w_1 = g$. We found that the bits in $C_4 = ((1000) \xrightarrow{\ll} 1) \& (0001) = (0001)$ are not all 0's and $j = m = 4$. We report the existence of an exact match and set $|LSP_c(W, P)| = 4$. Then the shift $S = m - (j - 1) = 4 - 3 = 1$, which is shown in Figure 8.

In the third attempt, $j = 1$ and we scan the characters of window $W = \text{tcac}$ from right to left. We start with the rightmost character $w_4 = c$. The process for the first iteration in the third attempt is shown in Figure 9. In Figure 9, we get

$$\begin{array}{lcl}
P = \text{tcag} & j = 4 & W = \text{gtca} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array} & \begin{array}{l} T = \text{atgg} \overline{\text{tca}} \\ C_3 = 1000 \xleftrightarrow{\leq 1} \\ C'_3 = 0001 \\ \& B_c[g] = 0001 \\ \hline C_4 = 0001 \\ \text{Report a match} \\ |LSP_c(W, P)| = 4 \end{array} &
\end{array}$$

Figure 8: The fourth iteration in the second attempt.

$$\begin{array}{lcl}
P = \text{tcag} & j = 1 & W = \text{tcac} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array} & \begin{array}{l} T = \text{atgg} \overline{\text{tca}} \\ C_0 = 1111 \xleftrightarrow{\leq 1} \\ C'_0 = 1111 \\ \& B_c[c] = 0100 \\ \hline C_1 = 0100 \end{array} &
\end{array}$$

Figure 9: The first iteration in the third attempt.

$C_1 = (0100)$ and $c_2 = 1$. Thus we have $j = 1$ and $i = 2$. This means that the suffix w_4 is equal to the prefix of $P^2 = \text{cagt}$ with length $j = 1$.

In the second iteration, $j = 2$, we have $C_1 = (0100)$ and the character $w_3 = \text{a}$. After bit-rotation operation and AND operation, we found that $C_2 = ((0100) \xleftrightarrow{\leq 1}) \& (0010) = (0000)$ and then we stop the process. The shift $S = 4 - (j - 1) = 4 - 1 = 3$, which is shown in Figure 10. After shifting, the boundary of the window is larger than n . We stop.

Next, we show the correctness of Algorithm 3. It is not difficult to see that the following loop invariant property holds at Line 11 in Algorithm 3, by setting $\ell = m - j$ and $k = ((i + \ell - 1) \bmod m) + 1$:

$$\text{Inv}(j) : c_k = 1 \text{ iff } P^i(\ell + 1, m) = W(\ell + 1, m) \quad (5)$$

for $1 \leq i \leq m$, where c_k is the k th bit of C_j . When $j = m$, $\text{Inv}(m)$ asserts $c_i = 1$ iff

$$\begin{array}{lcl}
P = \text{tcag} & j = 2 & W = \text{tcac} \\
\begin{array}{|l|} \hline B_c[t] = 1000 \\ B_c[c] = 0100 \\ B_c[a] = 0010 \\ B_c[g] = 0001 \\ \hline \end{array} & \begin{array}{l} T = \text{atgg} \overline{\text{tca}} \\ C_1 = 0100 \xleftrightarrow{\leq 1} \\ C'_1 = 1000 \\ \& B_c[a] = 0010 \\ \hline C_2 = 0000 \end{array} &
\end{array}$$

Figure 10: The second iteration in the third attempt.

$P^i(1, m) = W(1, m)$. It indicates there is a match (cf. Line 12). When $C_j = 0^m$ for some j , it implies there is no match (cf. Line 11).

6 The Time-Complexity Analysis

The time complexity of CBNDM is analyzed as follows. We first consider its worst-case time complexity. The preprocessing phase, which computes the B_c incidence matrix, takes $O(rm)$ time. If the alphabet size r is a constant, it is reduced to $O(m)$. In the searching phase, there are at most $O(n)$ sliding windows in worst case since the shift of a sliding window can be as small as 1. For each sliding window, at most m characters on it can be scanned from the right end to the left end. Therefore there are at most $O(m)$ iterations of Line 10 in Algorithm 3. Each execution of Line 10 takes $O(\lceil m/\kappa \rceil)$ time. Combining all of these facts together, we have $O(n) \times m \times \lceil m/\kappa \rceil = O(nm \lceil m/\kappa \rceil)$ time in worst case. The worst case happens when the text is a^n and the pattern is a^m . Notice that the worst case time complexity for MultiBNDM solving the circular pattern matching problem is $O(nm \lceil m^2/\kappa \rceil)$, which is an order of magnitude slower than CBNDM.

The worst-case time complexity for CBNDM seems very high, as compared to other text processing algorithms. However, this does not imply that CBNDM is impractical and inefficient. Let us consider average-time complexity for CBNDM. Since the analysis is exactly the same as BNDM shown in [13], we only give a sketch of the analysis and omit the details in this paper. The average shift for a sliding window is $\Theta(m)$, which implies that there are $O(n/m)$ sliding windows on the average. The average number of characters scanned in a sliding window is $O(\log_r m)$. During each scan, it takes $O(\lceil m/\kappa \rceil)$ time to process bit-parallel operations. Therefore, the average-time complexity for CBNDM is $O(n \log_r m / m \times \lceil m/\kappa \rceil)$, which is asymptotic to $O(n \log_r m / \kappa)$. The term $\log_r m$ is usually very small. Due to the simplicity of CBNDM, the ignored constant coefficient in the asymptotic bound is relatively small. In our case, κ is 32, which contributes totally to the speedup. This is why CBNDM is efficient. It is easy to see that the space complexity of CBNDM is $O(m)$ if $r = O(1)$.

7 Experimental Results

In this section, we give experimental results to demonstrate the practicability of the CBNM algorithm. The experiments are conducted as follows. We maintain two sets of testing data, one for random text searching and the other for DNA sequence searching. We also implemented the CBNM algorithm and the MultiBNM algorithm, which are both written in C, compiled by GCC compiler and executed on a PC with 3Ghz Intel CPU, 1GB memory installed Fedora 6 OS. Notice that a machine word has 32 bits in our environment, and this parameter is crucial for practicing bit-parallelism.

We use the overall execution time to measure the performance of the algorithms. Since there are other factors, such as multitasking and caching, that may affect the execution time, we repeat each measurement 10 times and then take their average in our final report. We are also interested in another measure that reflects the average number of bit-parallel operations needed for processing one character on the text. This measure is got by recording the number of executions of Line 13 in Algorithm 2 and Line 10 in Algorithm 3, and then dividing this number by the length of the text. Let us name this measure the CPS (Comparisons Per Site on the text). Notice that CPS is irrelevant to the length of the machine word, and in fact, it should have the same value for both CBNM and MultiBNM. In order to adjust them to reflect the fact that a machine word is limited to 32 bits, we shall multiply the CPS value by the length of the bit vector and then divide it by 32. Let us refer to this measure as the NCPS (Normalized CPS).

As discussed in Sect. 6, CBNM and MultiBNM are both factor-based algorithms, and they are especially efficient when dealing with random text searching. However, even under this preference, CBNM should perform much better than MultiBNM because CBNM has smaller pattern size to process, for the simple reason that it does not concatenate all circular strings altogether as what MultiBNM does. This is the reason why CBNM is more efficient than MultiBNM while dealing with circular pattern matching.

7.1 An Experiment with Random Text Searching

In our first experiment, we investigate on the performance of CBNM and MultiBNM when dealing with random texts. We set parameters

$n = 10^6$, $m \in \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$ and the alphabet size $r \in \{2, 4, 8, 16\}$. For each combination, we generate 100 random samples. That is, a length m pattern and a length n text are composed from a uniformly distributed random source with r symbols for each testing sample. Each sample is processed and repeated 10 times by CBNM and MultiBNM, in order to get more accurate execution time.

The final results are shown in Tables 1 and 2. Table 1 lists the execution time of these two algorithms and Table 2 shows their NCPS values. The first columns in both tables indicate the pattern lengths of the testing samples. The second and third columns list the results for CBNM and MultiBNM with different alphabet sizes and pattern lengths. Their performance ratios are calculated in the fourth columns. As we can see in Table 1, CBNM runs much faster than the MultiBNM. From the discussion in Sect. 6, we know that CBNM is an order of magnitude faster than MultiBNM in both worst case and average case. This fact is approved by our first experimental results. Notice that the performance ratios in Table 1 are a little smaller than the corresponding entries in Table 2. This is because additional execution time is spent for lines other than line 13 in Algorithm 2 and line 10 in Algorithm 3. Hence, the ratios of execution time would become rather mild.

7.2 An Experiment with DNA Pattern Searching

In our second experiment, we test circular pattern searching for DNA sequences. We choose the top 5 longest chromosome contigs from humans as the texts. Their NCBI reference numbers, as well as the contig lengths, are shown in Table 3. We extract the first 100 nucleotides of NT.023603.5 as the pattern throughout this experiment. Notice that the alphabet size is always 4. The final results are shown in Table 4. Notice that the performance ratios are almost a constant. This is because the pattern length m is fixed to 100. Another interesting point is to compare the NCPS values in Table 4 with Table 2, which is for random cases. From Table 2, if $m = 100$ and $r = 4$, we have NCPS = 0.1751 for CBNM and 13.6987 for MultiBNM. The ratio of this two value is 78.25, which is exactly the same as the NCPS ratio in Table 4. The NCPS values in Table 4 are around 1.5 for CBNM and 12 for MultiBNM, and these values are slightly better than the ran-

Table 1: The execution time (in seconds) for searching circular random patterns in a text.

m	CBNDM(A)				MultiBNDM (B)				Ratio =(B)/(A)			
	2	4	8	16	2	4	8	16	2	4	8	16
10	0.0322	0.0133	0.0091	0.0074	0.0755	0.0305	0.0220	0.0172	2.3425	2.2969	2.4189	2.3374
20	0.0140	0.0073	0.0051	0.0042	0.1011	0.0491	0.0364	0.0294	7.2223	6.7174	7.0927	6.9401
30	0.0094	0.0052	0.0036	0.0030	0.1348	0.0697	0.0506	0.0415	14.3343	13.3151	13.8686	13.9045
40	0.0110	0.0058	0.0041	0.0034	0.1715	0.0882	0.0642	0.0527	15.6128	15.1366	15.6988	15.7385
50	0.0089	0.0049	0.0034	0.0028	0.2141	0.1130	0.0804	0.0663	24.0669	23.2289	23.4783	23.8509
60	0.0077	0.0042	0.0030	0.0024	0.2571	0.1351	0.0964	0.0793	33.4943	32.1156	32.5315	32.9631
70	0.0085	0.0047	0.0034	0.0028	0.3018	0.1587	0.1136	0.0928	35.6913	33.6740	33.2804	33.6239
80	0.0076	0.0042	0.0031	0.0025	0.3476	0.1811	0.1308	0.1065	45.6773	42.8481	42.4240	43.1687
90	0.0068	0.0039	0.0028	0.0022	0.3909	0.2070	0.1486	0.1202	57.2860	53.4439	52.6864	53.5163
100	0.0079	0.0044	0.0032	0.0026	0.4392	0.2325	0.1673	0.1350	55.6568	52.7903	51.6198	52.3348

Table 2: The NCPS values for searching circular random patterns in a text.

m	CBNDM(A)				MultiBNDM (B)				Ratio =(B)/(A)			
	2	4	8	16	2	4	8	16	2	4	8	16
10	0.8361	0.3148	0.2072	0.1588	3.3544	1.2591	0.8289	0.6353	4.0120	4.0000	4.0000	4.0000
20	0.3733	0.1722	0.1209	0.0943	4.8529	2.2384	1.5719	1.2264	13.0000	13.0000	13.0000	13.0000
30	0.2541	0.1237	0.0847	0.0677	7.3688	3.5865	2.4566	1.9642	29.0000	29.0000	29.0000	29.0000
40	0.3911	0.1920	0.1328	0.1068	9.7777	4.7997	3.3207	2.6698	25.0000	25.0000	25.0000	25.0000
50	0.3167	0.1592	0.1098	0.0878	12.5107	6.2884	4.3376	3.4691	39.5000	39.5000	39.5000	39.5000
60	0.2698	0.1359	0.0942	0.0749	15.2457	7.6784	5.3225	4.2324	56.5000	56.5000	56.5000	56.5000
70	0.3534	0.1781	0.1239	0.0979	18.1432	9.1435	6.3583	5.0256	51.3333	51.3333	51.3333	51.3333
80	0.3144	0.1587	0.1109	0.0869	20.9591	10.5771	7.3935	5.7946	66.6667	66.6667	66.6667	66.6667
90	0.2821	0.1441	0.1002	0.0783	23.8856	12.2007	8.4853	6.6308	84.6667	84.6667	84.6667	84.6667
100	0.3447	0.1751	0.1225	0.0951	26.9754	13.6987	9.5826	7.4383	78.2500	78.2500	78.2500	78.2500

Table 3: The top 5 longest chromosome contigs of the human chromosome.

NCBI reference	Length
NT_005403.17	84213159
NT_026437.12	88289540
NT_032977.9	90908613
NT_005612.16	100537107
NT_016354.19	115591997

dom cases in Table 2. This suggests that CBNDM may perform better on DNA sequence searching than random text searching. At least, they are competitive.

8 Conclusion

In this paper, we propose the circular-pattern matching problem, as well as a bit-parallel algorithm named CBNDM for solving it. The performance of CBNDM is an order of magnitude faster than the MultiBNDM for solving the circular-pattern matching problem. The time complexity of CBNDM is $O(nm \lceil m/\kappa \rceil)$ in worst case, and it is $O(n/m \times \lceil \log_r m \rceil \times \lceil m/\kappa \rceil)$ in average case. The experimental results in Sect. 7 also agree with the theoretical bound for the average cases. Finally,

we claim that the performance of CBNDM processing DNA sequence searching is as competitive as processing random text searching.

One might be curious about the performance of circular-pattern matching by using the Naïve algorithm, which expands a circular pattern into m ordinary patterns and then checks the occurrences of these patterns for every position on the text. A check terminates immediately when it failed. Table 5 shows parts of the experimental results for random patterns, in the same setting as for Table 1. It reveals that the Naïve algorithm is the poorest, as one might expect.

References

- [1] A. V. Aho and M. J. Corasick. Efficient string matching: an aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, 1975.
- [2] C. Allauzen and M. Raffinot. Factor oracle of a set of words. Technical report 99-11, Institut Gaspard-Monge, Université de Marne-la-Vallée, 1999.
- [3] R. A. Baeza-Yates and G. H. Gonnet. A new

Table 4: Results for DNA circular pattern searching.

NCBI reference	n	CBNDM(A)		MultiBNDM (B)		Ratio =(B)/(A)	
		NCPS values	Run time (Sec.)	NCPS values	Run time (Sec.)	NCPS ratio	Run time ratio
NT_005403.17	84213159	0.1541	0.290953	12.0607	15.753600	78.25	54.14
NT_026437.12	88289540	0.1526	0.302227	11.9389	16.536675	78.25	54.72
NT_032977.9	90908613	0.1526	0.312470	11.9427	16.829435	78.25	53.86
NT_005612.16	100537107	0.1540	0.347171	12.0517	18.798967	78.25	54.15
NT_016354.19	115591997	0.1551	0.401382	12.1367	21.750664	78.25	54.19

Table 5: The execution time (in seconds) of the Naïve algorithm for searching circular random patterns in a text.

m	Naïve (C)			
	2	4	8	16
10	0.1280	0.0849	0.0584	0.0474
20	0.2568	0.1696	0.1167	0.0947
30	0.3844	0.2542	0.1775	0.1419
40	0.5126	0.3388	0.2342	0.1892
50	0.6410	0.4237	0.2917	0.2364
60	0.7691	0.5082	0.3500	0.2836
70	0.8973	0.5925	0.4083	0.3308
80	1.0268	0.6782	0.4667	0.3779
90	1.1542	0.7629	0.5252	0.4253
100	1.2827	0.8470	0.5835	0.4725

approach to text searching. *Communications of the ACM*, 35(10):74–82, 1992.

- [4] A. Blumer, J. Blumer, A. Ehrenfeucht, D. Haussler, and R. McConnel. Complete inverted files for efficient text retrieval and analysis. *Journal of the ACM*, 34(3):578–595, 1987.
- [5] B. Commentz-Walter. A string matching algorithm fast on the average. In *Proceedings of the 6th Colloquium, on Automata, Languages and Programming*, pages 118–132, London, UK, 1979. Springer-Verlag.
- [6] M. Crochemore, A. Czumaj, L. Gasieniec, S. Jarominek, T. Lecroq, W. Plandowski, and W. Rytter. Speeding up two string-matching algorithms. *Algorithmica*, 12(4-5):247–267, 1994.
- [7] M. Crochemore, A. Czumaj, L. Gasieniec, T. Lecroq, W. Plandowski, and W. Rytter. Fast practical multi-pattern matching. *Information Processing Letters*, 71(3-4):107 – 113, 1999.
- [8] D. Gusfield. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
- [9] S. Kim and Y. Kim. A fast multiple string-pattern matching algorithm. In *Proceedings of 17th AoM/IAoM Conference on Computer Science*, 1999.
- [10] M. O. Külekci. Tara: An algorithm for fast searching of multiple patterns on text files. In *Computer and information sciences, 2007. iscis 2007. 22nd international symposium on*, pages 1–6, Nov. 2007.
- [11] R. C. T. Lee. String matching. (*Unpublished*), 2010.
- [12] G. Navarro and M. Raffinot. A bit-parallel approach to suffix automata: Fast extended string matching. In *Proceedings of the 9th Annual Symposium on Combinatorial Pattern Matching, LNCS v. 1448*, pages 14–33, 1998.
- [13] G. Navarro and M. Raffinot. Fast and flexible string matching by combining bit-parallelism and suffix automata. *ACM Journal of Experimental Algorithmics*, 5(4), 2000.
- [14] G. Navarro and M. Raffinot. *Flexible pattern matching in strings: Practical on-line search algorithms for texts and biological sequences*. cambridge University Press, 2002.
- [15] M. Raffinot. On the multi backward dawg matching algorithm (multibdm). In *R. Baeza-Yates, Editor, Proc. 4th South American Workshop on String Processing*, Carleton University Press, pages 149–165, 1997.
- [16] Y. Shiloach. Fast canonization of circular strings. *Journal of Algorithms*, 2(2):107 – 121, 1981.
- [17] S. Wu and U. Manber. A fast algorithm for multi-pattern searching. Technical report, Department of Computer Science, Chung-Cheng University, 1994.